Student name:_____ ID: _____

CSE 2046/2246 Analysis of Algorithms

Spring 2025 Midterm, duration: 105 minutes

Q1	Q2	Q3	Q4	SUM
/17	/26	/30	/32	/100

Q-1. (17 pts) True/False. For each of the following indicate whether it is true or false. Justify your answer. True answers without justification will get 1 point, and with wrong justification 0 point.

1.a. (3 pts) For binary search, worst case and average case time complexities can be expressed as Θ(logn), while the overall time complexity is O(logn), rather than Θ(logn).

True, because best case is $\Theta(1)$, $O(\log n)$ indicates that the time complexity is at most $\log n$, which is true.

1.b. (4 pts) $\Theta(f(m+n))$ is equivalent to $\Theta(\max\{f(m), f(n)\})$ for any function $f(\cdot)$.

False. For example, for $f(n) = n!$, $\Theta((n+2)!)$ is not equivalent to $\Theta(\max\{n!, n!\})$

1.c. (3 pts) For an algorithm, if the worst case time complexity is $\Theta(f(n))$ and the best case time complexity is $\Theta(g(n))$, then the average case time complexity is $\Theta((f(n)+g(n))/2)$

False. We need probability distribution of possible inputs to calculate the avg time complexity. There are many examples where avg case time complexity is equivalent to the worst case time complexity, but higher than the best case.

1.d. (3 pts) $n \log_2 n$ has lower asymptotical order than $\sqrt{n^3}$.

$$\lim_{n\to\infty} \frac{n \log n}{n \cdot \sqrt{n}} = \lim_{n\to\infty} \frac{\frac{1}{n}}{\frac{1}{2} n^{-\frac{1}{2}}} = 0$$

So it is True.

1.e. (4 pts) In an undirected graph, some of the vertices are defined as Super Nodes (SN). In order to find the closest SN to a given vertex, we may apply graph traversal algorithms, and depth first search is more suitable than breadth first search.

Yes we can apply graph traversal algorithms, but bfs is more suitable than dfs, because bfs also finds the min-hop distance. When we apply bfs, the first visited SN would be the answer. So, False

Student name:_____ ID: _____

Q-2. (26 pts) Consider the following function.

```
int func(int A[l..r]) {
if (|l − r| <= 4)
      return 2;
else {
      int a = 1;
      for (int i=0,i<4;i++){
            l = ⌊i ∗ n/4⌋;
            r = ⌊(i + 1) ∗ n/4⌋;
            a = a * func(A[l..r]);    //line 1
      }
      return a;
}
```

(a – 8 pts) Find a recurrence relation for number of multiplications perfomed in line 1, when the input array size is n. Do not forget the base case. Also neglect multiplication by 1.

$M(n) = 0$ if $n \leq 4$

$M(n) = 4M\left(\frac{n}{4}\right) + 3$ if $n > 4$

(b – 13 pts) Solve the recurrence relation using backward substitition. (Find the exact value. You may assume that n is a power of 4)

$$M(4^k) = 4M(4^{k-1}) + 3$$

$$= 4\left(4M(4^{k-2}) + 3\right) + 3 = 4^2 M(4^{k-2}) + 4 \cdot 3 + 3$$

$$= 4^2\left(4M(4^{k-3}) + 3\right) + 4 \cdot 3 + 3 = 4^3 M(4^{k-3}) + 4^2 \cdot 3 + 4 \cdot 3 + 4$$

$$\vdots$$

$$= 4^{k-1} M(4^{k-(k-1)}) + \left(4^{k-2} + 4^{k-3} + \dots + 4^1\right) \cdot 3$$

$$= 0 + \left(\frac{4^{k-1} - 1}{4-1}\right) \cdot 3 = \frac{n}{4} - 1 \in \Theta(n)$$

(c – 5 pts) Give a solution using Master Theorem.

$a = 4 \quad b = 4 \quad d = 0$

$a > b^d$

Therefore $M(n) \in \Theta\left(n^{\log_4 4}\right) = \Theta(n)$ //

If $a < b^d$,	$T(n) \in \Theta(n^d)$
If $a = b^d$,	$T(n) \in \Theta(n^d \log n)$
If $a > b^d$,	$T(n) \in \Theta(n^{\log_b a})$

Student name:_____ ID: _____

Q-3. (30 pts) An anagram is a rearrangement of the characters of a string. For example "eat" and "tea" are anagrams. Similarly, "Dormitory" and "Dirtyroom" are anagrams of each other. Given an array A that consists of n distinct strings $[a_1, a_2, a_3 \ldots, a_n]$ each with at most k characters, and another array B that consists of m distinct strings $[b_1, b_2, b_3 \ldots, b_m]$ each with at most k characters, the problem is to find the number of (a_i, b_j) pairs that are anagrams of each other.

***Notes:** Comparing two strings takes O(k) time. Thus, an algorithm that performs O(f(n,m)) string comparisons will require O(kf(n,m)) character comparisons. You may assume that both m and n are larger than k.*

a) **(15 pts)** First, assume that no anagrams exist within array A or B (i.e., all strings in A are distinct and non-anagrammatic, and the same applies to B). To find the number of anagram pairs between A and B, give step-by-step description of an algorithm based on sorting and searching. Describe its time complexity in terms of m, n, k.

Two strings are anagrams if their sorted characters are equal.

Step 1: For each string in A, sort its characters and store the result in Sorted_A. $O(nk\log k)$

Step 2: Sort the Sorted_A array lexicographically $O(k \cdot n\log n)$

Step 3: For each string in B,
Sort its characters to get a key
Use binary search on sorted_A to check if the key exists
If it exists, increment the count.
} $O(mk \cdot \log n)$

Overall time complexity: $O((n+m)k\log n)$

b) **(15 pts)** Now, assume that there may exist anagrams within array A or B (e.g., both "top" and "pot" may appear in A or B). To find the number of anagram pairs between A and B, give step-by-step description of an algorithm based on merging and sorting. Describe its time complexity in terms of m, n, k.

Step 1: For each string in A and B, sort the characters of the string, and store the results in sorted_A & sorted_B. $O(nk\log k) + O(mk\log k)$

Step 2: Combine the arrays in a merged list, also store the source of every string (A or B).

Step 3: Sort the merged list. $O(k \cdot (n+m)\log(n+m))$

Step 4: Traverse the merged list
- For each group of identical strings, do:
  Count how many came from A: say $count_A$
  Count how many came from B: say $count_B$.
  Then total anagram pairs from this group: $count_A \times count_B$.

Step 4 includes one linear pass: $O(k(n+m))$

Overall time complexity is $O(k(n+m)\log(n+m))$.

Student name:_____ ID: _____

Q-4. (32 pts) Given an array A[1..n] of n positive integers, a majority element is an element that appears more than ⌊n/2⌋ times in the array. Consider the FindMajority problem which aims to find the majority element. You may assume that a majority element always exists in the input array.

Example: If input is A = [3, 3, 5, 5, 2, 5, 5], the output would be "5" since it appears 4 times which is greater than the half of the array size.

(a – 7 pts) Describe a pre-sorting based algorithm to solve the FindMajority problem and give its time complexity.

Sort the array in nondecreasing order. Since the majority element appears more than $\lfloor \frac{n}{2} \rfloor$ times, it must occupy the middle position. So, pick the element at position $A(\lfloor \frac{n}{2} \rfloor)$ $O(n \log n) + O(1) = O(n \log n)$ time complexity.

(b – 18 pts) ***Divide-and-Conquer.*** **Describe a divide-and-conquer algorithm for the same problem. Provide a step-by-step description rather than a pseudocode. Please apply your algorithm on the array A = [3, 3, 5, 5, 2, 5, 5]. Also give the time complexity of your algorithm (after defining the recurrence relation).**

Step 1: If the array has one element, return it as majority.

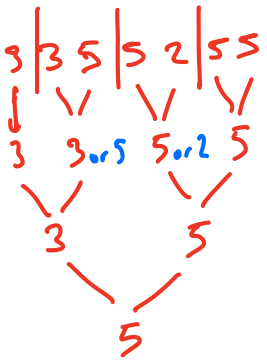
Step 2: Divide the array into two halves.

Step 3: Recursively find the majority in left half → m_L.

Step 4: Recursively find the majority in right half → m_R.

Step 5: If $m_L = m_R$ return m_L.

Step 6: If $m_L \neq m_R$, count m_L & m_R in the whole array, and return the one with higher count. (if equal, return any of them)



Time complexity: $T(n) = 2T(\frac{n}{2}) + O(n)$ $T(n) \in O(n \log n)$

(d – 7 pts) ***Problem Reduction.*** **Now consider another problem, namely MajorityExistence, defined as "Given an array A[1..n] of n positive integers, find whether there exists a majority element or not". This is a decision problem with a boolean output. Please reduce MajorityExistence problem to FindMajority problem, i.e. provide a solution using the algorithm of FindMajority. What is the time complexity of reduction?**

1. Call the FindMajority algorithm on array A. Let it return candidate element x.
2. Count the number of times x appears in A.
3. If count(x) > $\lfloor \frac{n}{2} \rfloor$, return true
   else return false.

Reduction takes $\Theta(n)$ time (counting step).